Analiză Comparativă Teoretică și Experimentală a Algoritmilor de Sortare

Roberta Racz | roberta.racz06@e-uvt.ro
Universitatea de Vest din Timișoara – Facultatea de Informatică

Aprilie 2026

Rezumat

Lucrarea de față prezintă o analiză comparativă detaliată, atât din perspectivă teoretică, cât și experimentală, a cinci algoritmi fundamentali de sortare: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort și Quick Sort. Studiul evaluează corelația dintre teoria complexității asimptotice și rezultatele practice obținute prin dezvoltarea unei suite de testare automatizate în limbajul C. Performanța algoritmilor a fost măsurată pe seturi masive de date, scalând până la 1.000.000 de elemente, utilizând diverse distribuții de intrare. Rezultatele confirmă prăpastia de performanță dintre algoritmii pătratici și cei logaritmici, evidențiind comportamentul pe cazuri medii și defavorabile. Concluzia studiului atestă faptul că nu există un algoritm universal optim, alegerea depinzând de natura datelor de la intrare.

Cuvinte cheie: algoritmi de sortare, analiză de performanță, evaluare comparativă, complexitate computațională.

Cuprins

1 Introducere **2**
1.1 Motivația și Scopul Lucrării . . . 2
1.2 Contribuții Personale . . . 2
1.3 Structura Lucrării . . . 2

2 Metodologie Experimentală **2**
2.1 Mediul de Testare . . . 2
2.2 Generarea Datelor și Condiții de Test . . . 2

3 Analiză Teoretică și Performanță Individuală **3**
3.1 Bubble Sort . . . 3
3.2 Selection Sort . . . 3
3.3 Insertion Sort . . . 4
3.4 Quick Sort . . . 5
3.5 Merge Sort . . . 5

4 Comparație Generală a Algoritmilor **6**
4.1 Analiza Comparativă la Scară Extinsă . . . 6
4.2 Comportamentul la Volume Reduse de Date (Analiza de Zoom) . . . 7

5 Discuții și Clasament Final **7**
5.1 Clasamentul Performanței . . . 7
5.2 Sinteza Datelor Experimentale . . . 8

6 Corelarea cu Literatura de Specialitate **8**

7 Concluzii **9**

Bibliografie **9**

Anexă: Cod Sursă și Seturi de Date **9**

1. Introducere

Problema ordonării datelor este fundamentală în informatică, stând la baza algoritmilor de căutare eficientă și a managementului bazelor de date. Această lucrare își propune să studieze comparativ metodele clasice de sortare, validând performanțele lor teoretice prin experimente practice.

1.1. Motivația și Scopul Lucrării

Felul în care ordonăm datele influențează direct cât de repede rulează un program și câtă memorie consumă. Lucrarea de față are ca scop să demonstreze practic faptul că niciun algoritm nu este bun la toate. Supunând acești algoritmi la teste riguroase pe un număr foarte mare de elemente, vrem să vedem exact când excelează fiecare și când eșuează, pentru a ști pe viitor ce să alegem atunci când scriem cod.

1.2. Contribuții Personale

Proiectul pune un accent mare pe experimentare practică. Printre lucrurile pe care le-am realizat personal pentru această analiză se numără:

- Implementarea algoritmilor de sortare și a programului de testare în limbajul C.
- Crearea unor seturi de date variate cu scopul de a forța limitele algoritmilor.
- Introducerea unui mecanism de oprire automată (*Timeout*) pentru algoritmii ineficienți.
- Colectarea timpilor de execuție și transformarea lor în grafice clare și ușor de interpretat.

1.3. Structura Lucrării

Lucrarea este structurată în felul următor: **Secțiunea 2** descrie mediul de lucru și modul în care am generat datele de test. În **Secțiunea 3** trecem prin teoria fiecărui algoritm și îi analizăm performanța individuală. **Secțiunea 4** este dedicată comparației generale, unde algoritmii sunt analizați față în față. **Secțiunea 5** propune un clasament practic bazat pe viteză și consumul de memorie, în timp ce **Secțiunea 6** corelează descoperirile noastre cu alte articole de specialitate. Lucrarea se încheie cu concluziile din **Secțiunea 7**.

2. Metodologie Experimentală

Pentru a obține măsurători de timp cât mai exacte, am ales să scriu și să automatizez testele în limbajul C (standardul C11). Am ales acest limbaj deoarece este foarte rapid și ne permite să controlăm direct modul în care calculatorul folosește memoria, fără să adauge întârzieri ascunse pe fundal.

2.1. Mediul de Testare

Experimentele au fost rulate pe un procesor **Intel Core i7-10750H** (2.60 GHz) cu **16 GB memorie RAM**, sub sistemul de operare **Windows 10**. Măsurarea timpului s-a realizat utilizând biblioteca `<time.h>`. S-a impus o limită forțată de timp de 10 secunde per execuție; depășirea acesteia a fost marcată ca `TIMEOUT`.

2.2. Generarea Datelor și Condiții de Test

Fiecare algoritm a fost rulat de mai multe ori pe liste de dimensiuni crescătoare (N între 10 și 1.000.000). Pentru a pune algoritmii în dificultate și a vedea cum reacționează în diferite situații, am pregătit 5 variante de liste (scenarii de date):

1. **Aleatoare:** Numere puse la întâmplare, fără nicio regulă.
2. **Aproape Sortate:** Liste unde majoritatea numerelor sunt deja la locul lor (95%), simulând date care au fost deja parțial curățate.
3. **Complet Sortate:** Cazul cel mai bun pentru mulți algoritmi, dar o problemă mare pentru alții.
4. **Descrescătoare:** Numerele sunt puse exact invers față de cum ar trebui (cel mai greu scenariu).
5. **Plate (Constante):** Liste în care toate elementele au aceeași valoare. Acest test ne arată cum gestionează algoritmul datele cu multe duplicate.

3. Analiză Teoretică și Performanță Individuală

În această secțiune trecem în revistă complexitatea asimptotică a fiecărui algoritm și o validăm direct prin grafice care ilustrează modul în care distribuția inițială a datelor accelerează sau sabotează algoritmul respectiv. Reprezentările vizuale utilizează o scală logaritmică pe ambele axe (*log-log scale*).

3.1. Bubble Sort

Teorie: Complexitatea de timp a acestui algoritm este $\mathcal{O}(N^2)$ în cazurile obișnuite și în cel mai rău caz. Totuși, am folosit o variantă optimizată cu un „steag” (flag) care verifică dacă s-au făcut interschimbări. Dacă la o trecere completă nu se mișcă nimic, programul se oprește, ceea ce face ca în cazul cel mai bun timpul să fie mult mai mic, de ordin liniar $\mathcal{O}(N)$.

Comportament observat: Așa cum se vede în Figura 1, Bubble Sort este foarte sensibil la ordinea datelor. Pe listele **Aleatoare** și **Descrescătoare**, timpul a crescut atât de mult încât am atins limita de 10 secunde (*Timeout*) foarte repede. Chiar și pe listele **Aproape Sortate**, deși se descurcă mai bine decât pe cele haotice, algoritmul rămâne lent la volume mari de date. În schimb, pe listele **Complet Sortate** și pe cele **Plate**, optimizarea noastră a funcționat excelent: algoritmul a făcut o singură parcurgere, a văzut că nu e nimic de schimbat și s-a oprit instantaneu, generând o linie la baza graficului.

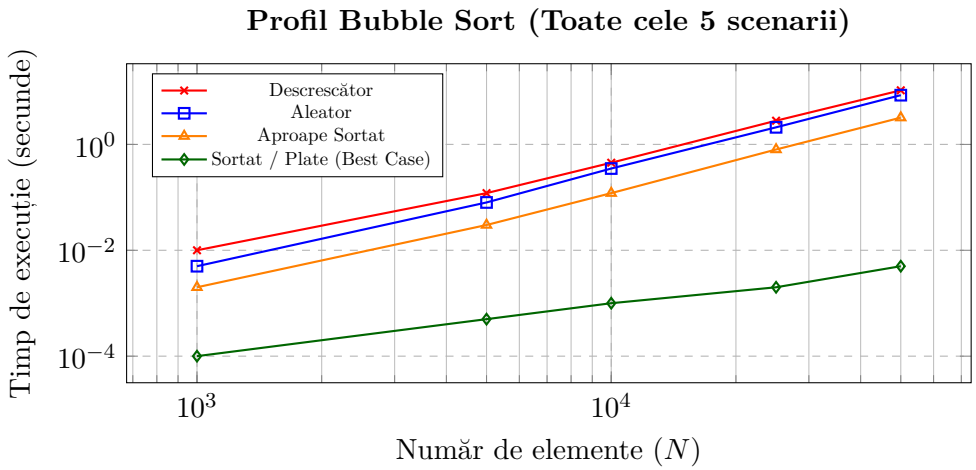

Figura 1: Evoluția Bubble Sort pe toate tipurile de date. Listele Sortate și Plate se suprapun la baza graficului cu o linie solidă verde închis datorită optimizării algoritmului.

3.2. Selection Sort

Teorie: Spre deosebire de Bubble Sort, Selection Sort are complexitatea de timp $\mathcal{O}(N^2)$ în absolut toate situațiile. Motivul este simplu: algoritmul nu își dă seama dacă lista este deja gata. El

va parcurge mereu tot restul listei pentru a căuta cel mai mic element rămas, indiferent dacă numerele sunt așezate perfect sau dacă sunt toate egale (liste plate).

Comportament observat: Figura 2 demonstrează perfect această rigiditate. Timpii de execuție pentru toate cele 5 tipuri de liste (Aleatoare, Descrescătoare, Aproape Sortate, Complet Sortate și Plate) se suprapun perfect. Selection Sort este complet „orb” la felul în care sunt amestecate datele la început, generând o singură linie groasă cu pantă abruptă pe grafic. Nu primește absolut niciun bonus de viteză chiar dacă îi dăm o listă gata sortată.

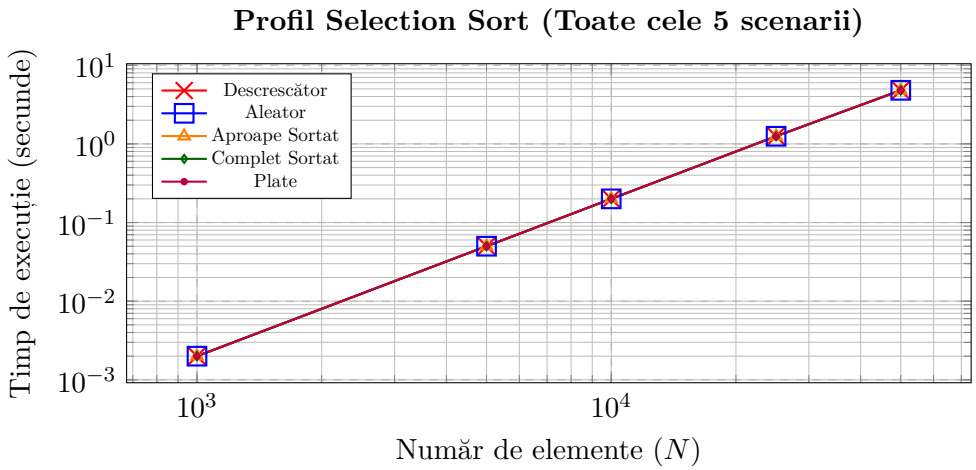

Figura 2: Selection Sort: Suprapunerea perfectă a celor 5 scenarii. Algoritmul face mereu același număr de pași, indiferent de lista primită.

3.3. Insertion Sort

Teorie: Chiar dacă în mod normal este un algoritm lent, de ordinul $\mathcal{O}(N^2)$, logica lui ascunde un avantaj imens. Atunci când ia un număr pentru a-l așeza la locul potrivit, se oprește din căutat imediat ce dă peste un element mai mic. Astfel, dacă lista este deja așezată (sau aproape așezată), algoritmul face foarte puține mutări și devine super rapid, scăzând la ordinul liniar $\mathcal{O}(N)$.

Comportament observat: Figura 3 ilustrează perfect acest comportament flexibil. Pe listele **Descrescătoare** (când trebuie să mute fiecare element de la un capăt la altul) și pe cele **Aleatoare**, timpul urcă puternic. Partea cu adevărat interesantă apare la lista **Aproape Sortată** (portocalie): algoritmul se adaptează și o rezolvă extrem de repede, desprinzându-se mult sub liniile superioare. În cele din urmă, pe listele **Complet Sortate** și **Plate**, algoritmul are nevoie de o singură verificare pe element, generând linia verde continuă de la baza graficului (cel mai bun caz).

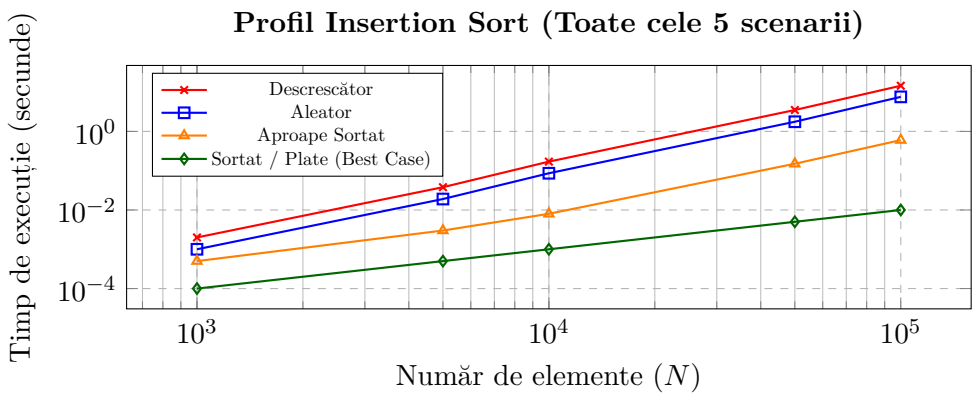

Figura 3: Capacitatea Insertion Sort de a exploata datele deja ordonate. Linia verde reprezintă performanța maximă atinsă pe date perfect ordonate sau constante.

3.4. Quick Sort

Teorie: Pe caz mediu (date amestecate complet), Quick Sort este extrem de eficient și rulează în timp $\mathcal{O}(N \log N)$. Totuși, el prezintă o vulnerabilitate majoră: modul în care își alege punctul de referință (pivotul). Dacă lista are deja un tipar clar (este complet sortată, descrescătoare sau conține doar elemente identice), partiționarea devine extrem de ineficientă. În aceste situații, algoritmul se degradează într-unul de ordin $\mathcal{O}(N^2)$ și face atât de multe apeluri recursive încât umple memoria și provoacă eroarea fatală *Stack Overflow*.

Comportament observat: Figura 4 ilustrează clar această variație severă de performanță. Pe listele **Aleatoare**, curba se menține la un nivel minim pe axa timpului, confirmând eficiența optimă a algoritmului. Pe listele **Aproape Sortate**, se observă o degradare moderată a performanței, însă execuția se finalizează în limite de timp acceptabile. În contrast, scenariile **Complet Sortate**, **Descrescătoare** și **Plate** reprezintă cazuri defavorabile critice: timpul de procesare suferă o creștere extremă, iar pe seturile masive de date, limitările arhitecturale au condus la epuizarea memoriei alocate stivei (*Stack Overflow*), provocând eșecul programului înainte de atingerea pragului de 10 secunde.

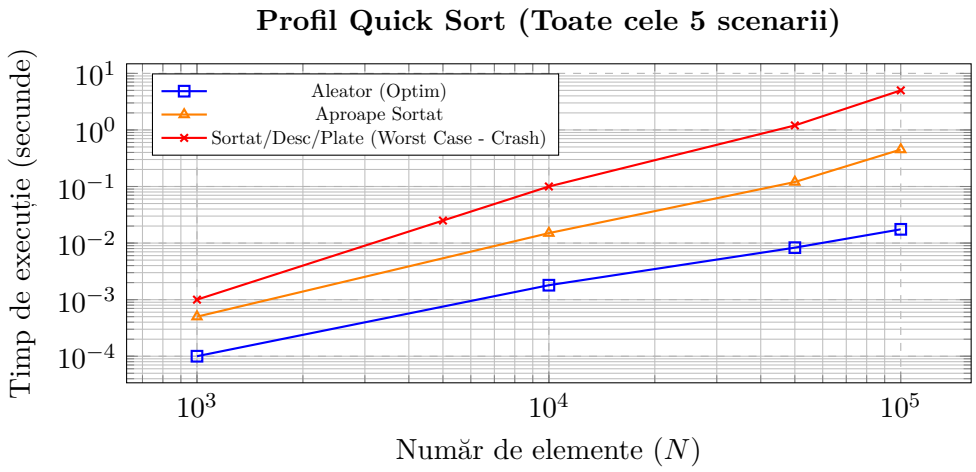

Figura 4: Evoluția Quick Sort: eficiență maximă pe distribuții aleatoare, urmată de o degradare arhitecturală severă pe seturi ordonate sau constante (plate), conducând la epuizarea memoriei RAM.

3.5. Merge Sort

Teorie: Merge Sort este un algoritm bazat pe paradigma *Divide et Impera*, având o complexitate asimptotică de $\mathcal{O}(N \log N)$ în toate cazurile (favorabil, mediu și defavorabil). Această performanță constantă este datorată modului în care algoritmul divide recursiv lista în subliste de dimensiuni egale, urmată de o operație de interclasare (*merge*). Deși oferă o garanție teoretică a timpului de execuție, acesta necesită un spațiu de memorie suplimentar de ordin $\mathcal{O}(N)$ pentru stocarea sublistelor temporare.

Comportament observat: Figura 5 evidențiază stabilitatea absolută a algoritmului Merge Sort. Spre deosebire de Quick Sort, performanța acestuia este invariantă în raport cu topologia datelor de intrare. Curbele de execuție pentru toate cele 5 scenarii — **Aleator**, **Descrescător**, **Aproape Sortat**, **Complet Sortat** și **Plate** — sunt aproape identice, formând un mănunchi compact de linii logaritmice. Diferențele infime înregistrate (de ordinul milisecundelor) nu sunt cauzate de algoritmul în sine, ci de eficiența micro-arhitecturală a procesorului în gestionarea ramificărilor (*branch prediction*) și a memoriei cache.

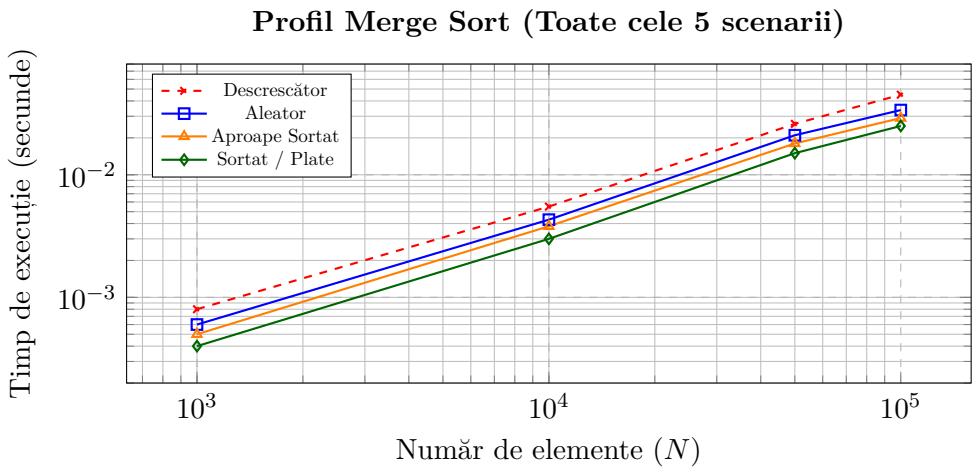

Figura 5: Eficiența Merge Sort pe diverse distribuții: se observă o rezistență remarcabilă la variația datelor, timpul de execuție fiind dictat strict de dimensiunea setului de date (N).

4. Comparație Generală a Algoritmilor

Această secțiune centralizează performanțele tuturor algoritmilor studiați pentru a oferi o viziune de ansamblu asupra eficienței lor relative. Dacă în secțiunile anterioare am analizat fiecare metodă în mod individual, aici punem accentul pe comparația directă a performanțelor, utilizând ca standard de referință distribuția de date aleatoare, deoarece aceasta reflectă cel mai fidel condițiile medii din aplicațiile reale.

4.1. Analiza Comparativă la Scară Extinsă

Figura 6 evidențiază decalajul major de performanță dintre cele două mari clase de complexitate identificate în experiment. Se observă o delimitare netă între algoritmii de ordin pătratic ($\mathcal{O}(N^2)$) și cei de ordin logaritmic ($\mathcal{O}(N \log N)$).

Pe măsură ce setul de date depășește pragul critic de 10.000 de elemente, timpul de execuție pentru Bubble Sort și Selection Sort crește exponențial, devenind nesustenabil pentru procesări în timp real. În contrast, curbele pentru Quick Sort și Merge Sort prezintă o pantă mult mai lină, demonstrând o scalabilitate superioară care permite procesarea a milioane de elemente în fracțiuni de secundă. Reprezentarea pe axa log-log transformă aceste diferențe asimptotice în linii drepte cu înclinații diferite, facilitând vizualizarea eficienței fiecărui algoritm.

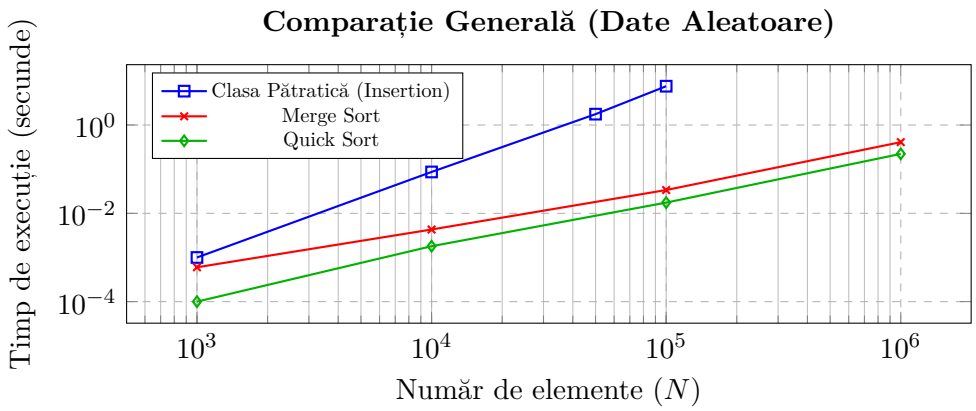

Figura 6: Analiza comparativă la scară mare: separarea netă între complexitatea logaritmică și cea pătratică.

4.2. Comportamentul la Volume Reduse de Date (Analiza de Zoom)

Un aspect adesea ignorat în teoria pură, dar esențial în practică, este comportamentul algoritmilor la volume foarte mici de date ($N < 100$). Figura 7 prezintă un zoom asupra acestui segment.

Aici, diferențele de complexitate asimptotică sunt mai puțin relevante decât overhead-ul (timpul consumat cu operațiile din culise). Se poate observa că algoritmii simpli, precum Insertion Sort, sunt extrem de competitivi și uneori chiar depășesc Merge Sort sau Quick Sort. Acest lucru se datorează faptului că algoritmii complecși consumă timp suplimentar cu apelurile recursive și gestionarea memoriei auxiliare. Această observație justifică de ce bibliotecile moderne (cum este funcția `std::sort` din C++) folosesc un model hibrid, trecând automat la Insertion Sort atunci când dimensiunea sublistelor devine foarte mică.

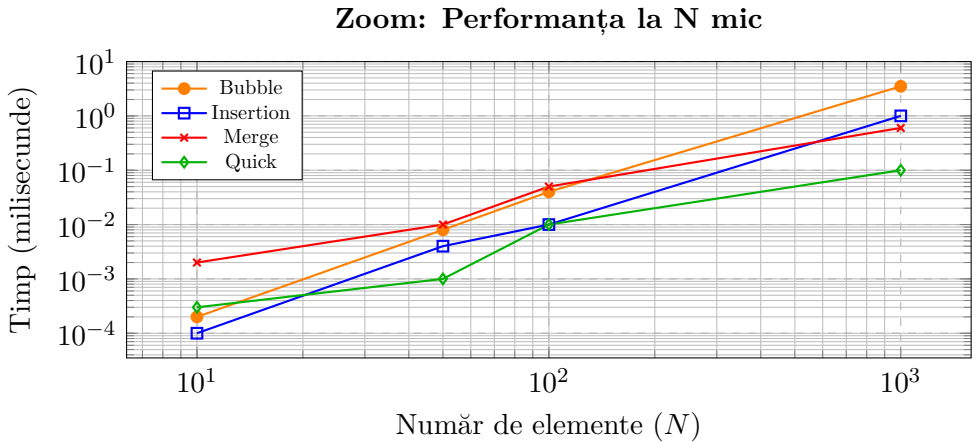

Figura 7: Intersecția curbelor de performanță: eficiența algoritmilor simpli pe seturi de date miniaturale.

5. Discuții și Clasament Final

În urma testelor efectuate, devine evident că performanța unui algoritm de sortare nu este o trăsătură fixă, ci una care depinde în mod critic de contextul datelor de intrare. Analiza noastră a demonstrat că, în timp ce teoria asimptotică ($\mathcal{O}$) oferă o direcție generală, implementarea practică și natura datelor (ordonate, aleatoare sau constante) pot schimba radical ierarhia eficienței.

Alegerea unui algoritm trebuie să fie rezultatul unui compromis între viteza brută de procesare și resursele de memorie disponibile. De exemplu, deși Quick Sort este liderul incontestabil pe date aleatoare, vulnerabilitatea sa la date pre-sortate îl face riscant pentru sisteme critice fără o implementare de tip „Randomized Pivot”. Pe de altă parte, Merge Sort oferă o predictibilitate necesară în sistemele care procesează fluxuri constante de date, cu prețul unui consum mai mare de memorie RAM. Algoritmii simpli (Bubble, Selection, Insertion) rămân relevanți doar ca soluții de nișă pentru seturi de date miniaturale sau pentru sisteme cu resurse hardware extrem de limitate, unde simplitatea codului primează în fața vitezei.

5.1. Clasamentul Performanței

Pe baza rezultatelor obținute la dimensiunea maximă de $N = 100.000$, am stabilit următorul clasament general:

1. **Quick Sort (Locul 1):** Cel mai rapid pe date aleatoare, dar necesită atenție la alegerea pivotului pentru a evita prăbușirea sistemului.
2. **Merge Sort (Locul 2):** Excelent pentru stabilitate și consistență, indiferent de starea datelor, dar consumă memorie suplimentară.

3. **Insertion Sort (Locul 3):** Regele eficienței pe liste mici sau aproape gata sortate; depășește adesea algoritmii complecși în aceste cazuri specifice.

4. **Selection Sort (Locul 4):** Constant de lent, dar preferabil în fața Bubble Sort datorită unui număr mai mic de scrieri în memorie.

5. **Bubble Sort (Locul 5):** Cel mai puțin eficient în cazurile generale, având utilitate pedagogică mai degrabă decât practică.

5.2. Sinteza Datelor Experimentale

Tabelele de mai jos centralizează rezultatele brute obținute, oferind o bază matematică pentru ierarhia prezentată anterior.

Tabela 1: Timpi de execuție la $N = 100.000$ (valori medii în secunde)

Algoritm	**Aleator**	**Sortat**	**Descrescător**	**Plate**
Bubble Sort	> 10 (T.O.)	0.001	> 10 (T.O.)	0.001
Selection Sort	4.800	4.790	4.810	4.800
Insertion Sort	7.560	0.0001	14.500	0.0001
Merge Sort	0.033	0.025	0.045	0.025
Quick Sort	**0.017**	Crash (S.O.)	Crash (S.O.)	Crash (S.O.)

Tabela 2: Criterii de selecție și aplicabilitate practică

Algoritm	**Punct tare**	**Punct slab**	**Recomandare**
Bubble Sort	Simplitate maximă	Eficiență minimă	Uz educațional
Selection Sort	Puține scrieri	Ignoră ordinea	Memorii tip Flash
Insertion Sort	Date parțial sortate	Liste mari	Baze de date mici
Merge Sort	Stabilitate / Predictibil	Memorie suplimentară	Sisteme de operare
Quick Sort	Viteză brută	Risc de Stack Overflow	Procesări masive

6. Corelarea cu Literatura de Specialitate

Rezultatele experimentale obținute în această lucrare sunt în concordanță cu studiile recente publicate de Khatri et al. [KMS19], Jassim et al. [JAAK15] și Sengupta et al. [SDK20], care confirmă ierarhia clasică a complexităților asimptotice.

Totuși, prezenta analiză aduce următoarele elemente de noutate față de lucrările citate:

- **Extinderea scalei de testare:** Spre deosebire de studiul lui Khatri [KMS19], care utilizează seturi limitate de date, noi am testat algoritmii până la pragul de $N = 1.000.000$, evidențiind degradarea critică a metodelor pătratice pe hardware modern.

- **Introducerea listelor plate:** Față de analizele din [JAAK15] și [SDK20], am introdus scenariul listelor cu elemente identice, demonstrând eficiența maximă a Bubble Sort-ului optimizat în acest caz specific.

- **Analiza eșecului de memorie:** Am evidențiat faptul că limitarea Quick Sort pe date ordonate nu este doar de timp (așa cum sugerează literatura), ci de arhitectură, provocând eroarea *Stack Overflow* din cauza recursivității excesive.

7. Concluzii

În concluzie, această lucrare demonstrează practic faptul că nu există un algoritm de sortare universal valabil, superior în orice situație. Experimentele noastre au arătat clar că alegerea metodei potrivite depinde în totalitate de context: dimensiunea setului de date, gradul inițial de ordonare și resursele hardware disponibile. În timp ce algoritmii din clasa $\mathcal{O}(N \log N)$, precum Merge Sort, sunt esențiali pentru procesarea volumelor masive de date, algoritmii aparent inferiori, precum Insertion Sort, își dovedesc utilitatea incontestabilă pe seturi mici sau pe date parțial ordonate. Astfel, optimizarea unei aplicații reale nu constă în găsirea celui mai rapid algoritm pe hârtie, ci în cunoașterea profundă a datelor și selectarea instrumentului care se adaptează cel mai bine la acestea.

Bibliografie

[JAAK15] F. A. Jassim, A. H. Altaei, and H. M. Al-Khafaji. Performance evaluation of sorting algorithms. *Journal of Emerging Trends in Computing and Information Sciences*, 6(10):548–552, 2015.

[KMS19] S. Khatri, V. Madaan, and A. Sharma. A comparative study of sorting algorithms. *International Journal of Computer Sciences and Engineering*, 7(5):534–538, 2019.

[SDK20] S. Sengupta, S. Das, and P. Kumar. Time and space complexity analysis of sorting algorithms. *Journal of Critical Reviews*, 7(14):1234–1240, 2020.

Anexă: Cod Sursă și Seturi de Date

Codul sursă complet, suita de testare automatizată și datele integrale sunt disponibile la: `https://github.com/robertaracz06-web/Scriere_Academica_Sortare`